

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Генетические алгоритмы

Студент гр. 4343

Г.Д. Маркашов

Студент гр. 4343

Д.Е. Селезнев

Студент гр. 4343

Р.А. Пимахин

Руководитель

Т.Р. Жангиров

Санкт-Петербург

2026

**ЗАДАНИЕ
НА УЧЕБНУЮ ПРАКТИКУ**

Студент: Г.Д. Маркашов

Студент: Д.Е. Селезнев

Студент: Р.А. Пимахин

Группа: 4343

Тема практики: Генетические алгоритмы

Задание на практику:

Дан взвешенный неориентированный граф (ребра имеют вес больше 0).
Необходимо найти минимальное остовное дерево для данного графа. С
использованием генетических алгоритмов

Ответственный за работу с данными, целостность доставки до пользователя
и алгоритмов - Г.Д. Маркашов

Ответственный за работу алгоритма - Д.Е. Селезнев

Ответственный за работу графической части ПО - Р.А. Пимахин

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 18.07.2026

Студент

Г.Д. Маркашов

Студент

Д.Е. Селезнев

Студент

Р.А. Пимахин

Руководитель

Т.Р. Жангиров

АННОТАЦИЯ

В данной работе рассматривается задача поиска минимального остовного дерева (МОД) во взвешенном неориентированном графе с использованием генетического алгоритма. Целью проекта является разработка программного приложения с графическим интерфейсом пользователя, позволяющего задавать исходные данные, настраивать параметры алгоритма, выполнять поиск решения и визуализировать процесс оптимизации.

В отчёте описываются особенности постановки задачи, структура представления графа, принципы работы реализованного генетического алгоритма, используемые операции формирования популяции, отбора, скрещивания и мутации, а также функция качества решения. Рассматривается архитектура разработанного программного обеспечения, возможности взаимодействия пользователя с графическим интерфейсом и способы визуализации процесса поиска минимального остовного дерева.

Результатом работы является программное приложение на языке Python, позволяющее находить приближённое решение задачи МОД, отображать промежуточные этапы работы алгоритма, строить график изменения качества решений по поколениям и сравнивать различные варианты полученных решений.

SUMMARY

In this paper, we consider the problem of finding a minimum spanning tree (MOD) in a weighted undirected graph using a genetic algorithm. The goal of the project is to develop a software application with a graphical user interface that allows you to set source data, configure algorithm parameters, search for solutions, and visualize the optimization process.

The report describes the specifics of the problem statement, the structure of the graph representation, the principles of operation of the implemented genetic algorithm, the operations of population formation, selection, crossing and mutation used, as well as the quality function of the solution. The architecture of the developed software, the possibilities of user interaction with a graphical interface, and ways to visualize the process of searching for a minimum spanning tree are considered.

The result of the work is a Python software application that allows you to find an approximate solution to the MOD problem, display the intermediate stages of the algorithm, graph changes in the quality of solutions over generations, and compare different variants of the solutions obtained.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 ПОСТАНОВКА ЗАДАЧИ	8
1.1 Предметная область	8
1.2 Задачи практики	8
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	10
2.1. Работа с данными. Проверка целостности и доставка данных от UI до алгоритма и обратно.	10
2.2. Генетический алгоритм для поиска МОД	12
2.3. Графический интерфейс пользователя (GUI) и визуализация	15
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	20
4 ОТЗЫВ РУКОВОДИТЕЛЯ	21
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24

ВВЕДЕНИЕ

Предметной областью данной работы являются задачи оптимизации на графах, в частности задача поиска минимального остовного дерева (МОД) во взвешенном неориентированном графе. Подобные задачи находят применение при проектировании сетей связи, транспортных систем, электрических сетей и других областях, где требуется построение наиболее эффективной структуры при минимальных затратах. Для решения сложных оптимизационных задач могут применяться эвристические методы, одним из которых являются генетические алгоритмы, основанные на моделировании процессов естественного отбора и эволюции.

Целью практики является разработка программного приложения для поиска минимального остовного дерева во взвешенном неориентированном графе с использованием генетического алгоритма. Разрабатываемая программа должна обеспечивать возможность задания исходных данных различными способами, настройки параметров алгоритма пользователем, визуального отображения процесса поиска решения и анализа эффективности работы алгоритма.

Для достижения поставленной цели необходимо решить следующие задачи: изучить особенности задачи поиска минимального остовного дерева и методы её решения; разработать представление графа и структуру генетического алгоритма; реализовать основные этапы работы алгоритма, включая формирование начальной популяции, оценку качества решений, отбор, скрещивание и мутацию; разработать графический интерфейс пользователя на языке Python; реализовать возможность ввода данных через интерфейс, загрузки из файла и генерации случайных графов; обеспечить пошаговую визуализацию процесса поиска решения, построение графика изменения функции качества и возможность получения итогового результата без просмотра всех промежуточных этапов.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Предметной областью практики являются поиск минимального остовного дерева во взвешенном неориентированном графе с использованием генетических алгоритмов. Подобные задачи применяются при проектировании сетей связи, транспортных систем, электрических сетей и других структур, где требуется найти наиболее эффективное соединение объектов с минимальными затратами.

В рамках практики рассматривается применение генетических алгоритмов для решения задач. Разрабатываемое программное приложение позволяет исследовать процесс поиска решения, настраивать параметры алгоритма и визуализировать этапы его работы.

1.2 Задачи практики

В рамках прохождения практики были поставлены следующие задачи:

1. Разработать слой работы с данными, обеспечивающий валидацию и целостность информации о графах, истории эволюции и параметрах алгоритма. (Маркашов Г.Д.)

2. Реализовать механизмы загрузки графов из файлов, генерации случайных связных графов и конвертации между различными представлениями данных. (Маркашов Г.Д.)

3. Реализовать алгоритмы кодирования и декодирования деревьев с использованием кодов Прюфера для представления решений в генетическом алгоритме. (Маркашов Г.Д., Селезнев Д.Е.)

4. Реализовать генетические операторы: несколько вариантов селекции, скрещивания и мутации с возможностью выбора типа оператора пользователем. (Селезнев Д.Е.)

5. Разработать генетический алгоритм для поиска минимального остовного дерева, включающий функцию приспособленности с системой штрафов, механизм элитизма и критерий сходимости. (Селезнев Д.Е.)

6. Реализовать механизм сохранения истории эволюции с возможностью отката к предыдущим поколениям и сбор статистики для анализа работы алгоритма. (Селезнев Д.Е., Маркашов Г.Д.)

7. Разработать модульный графический интерфейс пользователя с панелями для настройки параметров, управления процессом эволюции и отображения статистики. (Пимахин Р.А.)

8. Реализовать несколько способов создания и загрузки графов через графический интерфейс: из файла, генерацию случайных графов и ручной ввод. (Пимахин Р.А.)

9. Реализовать визуализацию графа с выделением найденного минимального остовного дерева и возможностью просмотра различных решений из популяции. (Пимахин Р.А.)

10. Реализовать график сходимости алгоритма для визуального анализа динамики оптимизации в процессе эволюции. (Пимахин Р.А., Маркашов Г.Д.)

11. Обеспечить интерактивное управление процессом эволюции: пошаговое выполнение, откат назад, автоматическое проигрывание и быстрый переход к результату. (Пимахин Р.А., Селезнев Д.Е.)

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Работа с данными. Проверка целостности и доставка данных от UI до алгоритма и обратно.

Класс `GAStateStep(BaseModel)` предназначен для хранения информации о текущем шаге эволюции и содержит следующие поля: `generation` – номер поколения, `population` – список хромосом (кодов Прюфера), `best_fitness` – лучшее значение качества в поколении, `avg_fitness` – среднее значение качества в поколении, `worst_fitness` – худшее значение качества в поколении.

Класс `GAHistory(BaseModel)` реализует хранение истории работы алгоритма и включает поля: `generations_count` – количество поколений, `best_fitness_history` – список лучших значений качества по поколениям, `avg_fitness_history` – список средних значений качества, `worst_fitness_history` – список худших значений качества, `steps` – список шагов эволюции (объектов `GAStateStep`). Для управления историей реализованы методы: `add_step` для добавления шага, `get_step` для получения шага по индексу, `get_last_step` для получения последнего шага.

Класс `GraphData(BaseModel)` служит для хранения информации о графе с полями `num_vertices` – количество вершин и `matrix` – матрица смежности. Для обеспечения корректности данных реализован метод `validate_matrix`, который срабатывает автоматически после создания объекта и выполняет проверки: отсутствие петель (при обнаружении выбрасывается `ValueError`) и использование только ориентированного графа (при нарушении выбрасывается `ValueError`). Для удобства конвертации данных реализованы методы `from_edges_list(n, edges)` для создания объекта из списка рёбер и `to_edges_list` для обратного преобразования в список рёбер.

Класс `Graph` предназначен для управления данными графа, их извлечения и применения в алгоритме. Конструктор принимает количество вершин и создаёт объект `GraphData`. Для работы с графом реализованы методы: `get_edges` – возврат списка рёбер, `get_matrix` – возврат матрицы смежности, `get_data` – возврат данных графа. Защищённые методы

`_build_adj_list` формируют список смежности, `_validate_vertex` проверяют вершину на допустимый диапазон. Методы управления рёбрами включают `add_edge` – добавление ребра, `find_edge` – поиск ребра, `has_edge` – проверка существования ребра, `get_edge_weight` – получение веса ребра. Для анализа графа реализованы: `get_copy_edges` – получение копии списка рёбер, `get_copy_neighbors` – копия списка смежности, `get_degree` – степень вершины, `get_total_edges` – количество рёбер, `get_total_weight` – суммарный вес всех рёбер, `get_edges_weight` – вес переданных рёбер. Методы `to_dict` и `to_json` обеспечивают миграцию данных графа в словарь и JSON-файл соответственно. Проверка свойств графа выполняется методами `is_connected` для связности и статическим методом `is_tree` для проверки списка рёбер на образование дерева.

Класс `PruferCode` реализует кодирование и декодирование графа в код Прюфера. Статический метод `edges_to_code` преобразует список рёбер в код Прюфера, метод `code_to_edges` выполняет обратное декодирование кода в список рёбер, а метод `is_valid_prufer_code` проверяет валидность кода Прюфера.

Класс `FileIO` обеспечивает работу с файлами для чтения графа из файла в формате списка рёбер. Статический метод `load` считывает данные из файла, выполняет их валидацию, создаёт объект класса `Graph` и возвращает его.

Класс `GraphGenerator` предназначен для генерации графа с заданными параметрами. Статический метод `gen_graph` принимает количество вершин, вероятность появления ребра между вершинами, минимальный и максимальный веса рёбер и генерирует граф в соответствии с этими параметрами.

Класс `GraphManager` представляет собой абстракцию, объединяющую несколько классов для удобного управления графом и валидации входных данных. Конструктор принимает объекты классов `FileIO` и `GraphGenerator`, а также содержит поле `_graph` для хранения графа. Методы управления графом: `get_graph` – возврат объекта графа при его наличии, `has_graph` –

проверка наличия графа, `set_graph` – установка графа, `clear` – удаление графа. Методы создания графа: `load_from_tuple` – создание графа из списка рёбер с весами, `generate_random_graph` – генерация случайного графа. Методы доступа к данным графа дублируют функциональность класса `Graph`, обеспечивая единый интерфейс управления.

Класс `HistoryManager` реализует хранение и управление историей эволюции особей и шагов алгоритма. Конструктор создаёт объект класса `GAHistory`. Методы управления историей: `add_steps` – добавление шага, `go_forward` – переход на следующий шаг, `go_back` – переход на шаг назад, `go_to` – переход на конкретный шаг, `go_to_end` – переход на последний шаг, `go_to_start` – переход в начало истории. Методы доступа к данным: `get_current` – извлечение данных текущего шага, `get_step` – получение шага по индексу, `get_all` – получение копии всей истории, `get_history_stats` – получение статистики, `get_last` – информация последнего шага, `get_first` – информация первого шага. Методы управления состоянием: `clear` – очистка истории, `is_empty` – проверка на пустоту.

Для избежания ошибок при конвертации параметров из графического интерфейса реализованы Enum-классы: `CrossoverType` – типы скрещивания, `MutationType` – типы мутации, `SelectionType` – типы селекции, обеспечивающие типобезопасность при передаче параметров в алгоритм.

Использование Pydantic-схем гарантирует строгую валидацию данных на всех этапах работы приложения. Модульная архитектура с разделением ответственности между классами обеспечивает гибкость и поддерживаемость кода: `GraphData` отвечает за хранение, `Graph` за управление, `GraphManager` за абстракцию, `HistoryManager` за историю, а Enum-классы гарантируют корректность типов при взаимодействии с пользовательским интерфейсом.

2.2. Генетический алгоритм для поиска МОД

Генетический алгоритм реализован в модуле `ga_engine.py` и включает два основных класса: `GeneticAlgorithmMST` для управления процессом эволюции и `GeneticOperators` для реализации генетических операторов.

Модуль operators.py

Класс GeneticOperators инкапсулирует все генетические операторы, обеспечивая модульность и возможность независимого тестирования. Метод create_random_individual генерирует случайные коды Прюфера заданной длины для инициализации популяции.

Турнирная селекция реализована методом tournament_selection, который случайно выбирает участников турнира через random.sample и находит победителя с минимальным fitness через функцию min, обеспечивая баланс между исследованием новых решений и эксплуатацией найденных. Рулеточная селекция в методе roulette_selection инвертирует fitness для задачи минимизации, создаёт накопительную сумму через sum и выбирает особь пропорционально её приспособленности.

Скрещивание реализовано тремя методами. Метод uniform_crossover на каждой позиции случайно выбирает ген от одного из родителей через цикл, обеспечивая равномерное смешивание генетического материала. Метод one_point_crossover выбирает случайную точку разреза через random.randint и обменивает хвостовые части родителей с помощью срезов списков, сохраняя большие блоки генов. Метод two_point_crossover выбирает две точки разреза и обменивает средние сегменты путём конкатенации трёх срезов, сохраняя граничные области от обоих родителей.

Мутация представлена двумя методами. Метод mutate проходит по всем позициям кода Прюфера и с заданной вероятностью заменяет каждый ген на новое случайное значение через random.randint, вводя новые гены в популяцию для увеличения разнообразия и избегания локальных оптимумов. Метод mutate_swap выбирает две случайные позиции через random.sample и обменивает гены местами через одновременное присваивание, сохраняя набор генов, но изменяя структуру кодируемого дерева.

Модуль ga_engine.py

Класс GeneticAlgorithmMST управляет полным циклом эволюции популяции решений, используя экземпляр класса GeneticOperators для

выполнения генетических операций. При инициализации через конструктор `__init__` класс принимает граф и параметры алгоритма, проверяет связность графа методом `is_connected`, вычисляет максимально возможный вес дерева для расчёта штрафов. Начальная популяция создаётся методом `_initialize_population`, который генерирует случайные коды Прюфера, оценивает их и сохраняет статистику поколения 0 в историю.

Центральным элементом алгоритма является функция приспособленности, реализованная методом `_calculate_fitness`. Она использует двухуровневую систему оценки: преобразует код Прюфера в рёбра дерева через `PruferCode.code_to_edges`, проверяет валидность каждого ребра методом `get_edge_weight` графа и возвращает суммарный вес для валидных деревьев или штрафное значение для невалидных. Штраф включает удвоенный максимальный вес дерева, тысячу за каждое невалидное ребро и суммарный вес валидных рёбер, что направляет эволюцию от невалидных решений через постепенное исправление ошибок к оптимальным.

Процесс эволюции реализован методом `step`, который выполняет одно поколение алгоритма. Сначала текущая популяция оценивается методом `_evaluate_population` для селекции родителей. Новая популяция формируется через элитизм, когда лучшие особи копируются без изменений, и генерацию потомков. Родители выбираются методом `_select_parent`, скрещиваются методом `_crossover` и мутируют методом `_mutate`, каждый из которых вызывает соответствующие методы класса `GeneticOperators` в зависимости от выбранного типа оператора. После создания новой популяции она оценивается повторно для сбора статистики, которая сохраняется методом `_save_state` в историю для возможности отката через метод `rollback`.

Метод `run` запускает цикл эволюции на заданное количество поколений, последовательно вызывая метод `step` для каждой итерации. Для работы с результатами реализованы методы `get_best_solution` для получения лучшего решения из текущей популяции, `get_all_solutions` для доступа ко всей отсортированной популяции с рангами, кодами Прюфера и весами особей, и

`get_statistics` для извлечения полной статистики по всем поколениям, включая списки лучшего, среднего и худшего `fitness`.

Использование кодов Прюфера гарантирует корректность всех генерируемых деревьев без циклов, устраняя необходимость проверок валидности на структурном уровне. Двухуровневая функция приспособленности с системой штрафов обеспечивает робастность алгоритма даже при случайной инициализации популяции с большим количеством невалидных особей.

2.3. Графический интерфейс пользователя (GUI) и визуализация

Графический интерфейс разработан с использованием стандартной библиотеки Tkinter, а также библиотек Matplotlib и NetworkX для динамической отрисовки графов и построения графиков сходимости. Интерфейс организован по модульному принципу, разделяя функциональность на компоненты, что обеспечивает удобство в расширении и внесении изменений.

Класс `GraphDrawer` предназначен для отрисовки графа и выделения найденного минимального остовного дерева. Конструктор класса принимает виджет `Canvas` из Tkinter, создаёт фигуру Matplotlib с белым фоном и интегрирует её в переданный контейнер через `FigureCanvasTkAgg`. Метод `draw` выполняет полную отрисовку графа: создаёт объект `networkx.Graph`, добавляет вершины и рёбра с весами, вычисляет стабильное расположение вершин с использованием алгоритма `spring_layout` (коэффициент `k` автоматически подбирается в зависимости от числа вершин для оптимального отображения). Для рёбер графа используется цвет `lightgray`. При передаче списка `mst_edges` метод выделяет рёбра минимального остовного дерева цветом `crimson` с увеличенной толщиной, а при передаче `selected_edges` подсвечивает рёбра выбранной особи зелёным цветом пунктирной линией, что позволяет визуально сравнивать различные решения. Метод `clear` полностью очищает координатные оси, сбрасывает кэш позиций вершин и обновляет полотно.

Класс `ParameterFrame` инкапсулирует все элементы управления для настройки параметров генетического алгоритма. Конструктор создаёт группу с рамкой, размещает поля ввода для следующих параметров: размер популяции (значение по умолчанию 100), количество поколений (200), вероятность скрещивания (0.8), вероятность мутации (0.05), метод селекции (выпадающий список с вариантами турнирной и рулеточной селекции), размер турнира (5), метод скрещивания (одноточечный, двухточечный, равномерный), метод мутации (обмен генов, замена генов) и количество лучших особей (2). Для обеспечения корректности взаимодействия метод `_update_selection` динамически управляет доступностью поля «Размер турнира»: при выборе рулеточной селекции поле блокируется, так как данный метод не требует указания размера турнира. Метод `get_parameters` считывает все значения из полей ввода, приводит их к соответствующим типам данных и возвращает словарь, готовый для передачи в генетический алгоритм.

Класс `ControlButtons` группирует все кнопки управления и обеспечивает взаимодействие пользователя с программой. Конструктор принимает функции-обработчики для каждого действия и размещает кнопки в три группы. Первая группа («Сгенерировать граф», «Загрузить граф», «Ввести вручную») отвечает за создание и загрузку графов. Вторая группа («Запустить алгоритм», «Сброс») управляет выполнением алгоритма. Третья группа («Шаг назад», «Авто-запуск», «Шаг вперёд», «Пропустить») обеспечивает пошаговое управление эволюцией. Метод `enable_controls` включает или отключает кнопки пошагового управления в зависимости от состояния алгоритма. Метод `enable_step_buttons` динамически управляет доступностью кнопок «Шаг назад» и «Пропустить» на основе текущего поколения: кнопка назад доступна при `generation > 1`, кнопка пропуска - при `generation < max_generations`. Метод `set_play_button_text` изменяет текст кнопки авто-запуска между «Авто-запуск» и «Пауза» в зависимости от режима воспроизведения.

Класс `StatisticsFrame` отвечает за отображение статистических метрик и построение графика сходимости. Конструктор создаёт группу с рамкой, размещает метки для отображения текущего поколения, лучшего, среднего и худшего значений функции приспособленности, а также веса минимального остова дерева. Метод `_create_plot` создаёт фигуру `Matplotlib`, настраивает оси с серым фоном, добавляет сетку и легенду. График содержит две визуальные компоненты: линию (`best_line`) зелёного цвета, показывающую динамику лучшего веса по поколениям, и точечный график (`scatter`) красного цвета, отмечающий поколения, в которых было найдено улучшение. Метод `update_stats` обновляет текстовые метки статистики на основе переданных данных. Метод `update_weight` обновляет вес МОД и добавляет точку на график, при этом ведётся учёт только корректных решений (вес которых не превышает штрафной порог), а при обнаружении улучшения точка добавляется на точечный график. Метод `truncate` обрезает данные графика до указанного поколения, что используется при откате алгоритма. Метод `bulk_update_weights` выполняет обновление графика при пропуске большого числа поколений. Метод `reset` полностью сбрасывает все метки статистики, очищает данные и обновляет график до начального состояния.

Класс `GA_GUI` является центральным классом главного окна программы и управляет всеми компонентами интерфейса, загрузкой данных и процессом эволюции. Конструктор инициализирует менеджер графа (`GraphManager`), устанавливает флаги состояния алгоритма и текущего шага, создаёт структуру интерфейса с левой панелью (граф, параметры, управление) и правой панелью (лог, статистика). Метод `_create_layout` формирует основную структуру окна: создаёт экземпляры всех компонентов (`ParameterFrame`, `ControlButtons`, `GraphDrawer`, `StatisticsFrame`, `ScrolledText`), настраивает их размещение и связывает события. Для работы с популяцией реализован выбор особей: метод `_populate_individual_selector` заполняет выпадающий список строками, а метод `_on_individual_selected` обрабатывает выбор пользователя, обновляет информационную метку и перерисовывает

граф с подсветкой рёбер выбранной особи. Метод `_redraw_with_selected` обеспечивает совместную отрисовку рёбер МОД и рёбер выбранной особи. Метод `load_in_ga` инициализирует объект генетического алгоритма с параметрами из интерфейса и проверяет наличие графа. Метод `reset` полностью сбрасывает состояние приложения: останавливает алгоритм, очищает холст, сбрасывает статистику и отключает кнопки управления. Метод `show_stats` обновляет статистику из генетического алгоритма, при обнаружении нового лучшего решения добавляет запись в лог. Метод `_is_valid_solution` проверяет корректность решения с использованием статического метода `Graph.is_tree`. Метод `update_plot` обновляет список особей, статистику и перерисовывает граф, а также управляет доступностью кнопок пошагового управления. Метод `run_algorithm` инициализирует и запускает алгоритм на одно поколение для демонстрации начального состояния. Метод `next_step` выполняет шаг вперёд, обновляет отображение и добавляет запись в лог. Метод `prev_step` выполняет откат на шаг назад с использованием `ga.rollback()` и синхронизирует график статистики. Метод `skip_to_end` выполняет пропуск к финальному поколению: отключает автопроигрывание, запускает `ga.run()` на оставшееся количество поколений, выполняет обновление графика через `bulk_update_weights` и отображает итоговое решение. Метод `toggle_play` переключает режим автопроигрывания, обновляет текст кнопки. Метод `play_loop` циклически выполняет `next_step()` с задержкой 500 мс.

Класс `GenerateGraphDialog` реализует диалоговое окно для генерации случайного графа с заданными параметрами. Конструктор создаёт окно с полями ввода для количества вершин (по умолчанию 20), вероятности появления ребра (0.3), минимального (1) и максимального (100) весов рёбер. Метод `_create` считывает значения из полей, преобразует их к соответствующим типам, вызывает функцию обратного вызова `on_generate` с переданными параметрами и закрывает окно.

Класс `ManualInputDialog` реализует диалоговое окно для ручного ввода графа в текстовом формате. Конструктор создаёт окно с инструкцией по вводу (каждое ребро на новой строке в формате «`u v w`») и текстовым полем с примером данных. Метод `_parse_and_save` обрабатывает введённый текст: разбивает на строки, каждую строку разделяет на три части, преобразует номера вершин в целые числа, а вес — в число с плавающей точкой. При обнаружении ошибок формата выводится сообщение с причиной ошибки. При успешной обработке вызывается функция обратного вызова `on_save` с полученным списком рёбер и окно закрывается.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

При выполнении практической работы использовались следующие технологии и программные средства:

- Python 3 основной язык программирования, на котором реализованы генетический алгоритм и программное приложение.
- Tkinter стандартная библиотека Python для создания графического интерфейса пользователя. Использовалась для разработки окон приложения, организации ввода исходных данных, настройки параметров генетического алгоритма и отображения результатов работы программы.
- NetworkX библиотека для работы с графами. Применялась для создания, визуализации взвешенного неориентированного графа.
- Rydantic библиотека для работы с данными, удобному обращению к ним и валидации их.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв

о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы была разработана полнофункциональная программная система для решения задачи поиска минимального остовного дерева взвешенного неориентированного графа с использованием генетического алгоритма.

Ядро системы представлено модулями генетического алгоритма и операторов, реализующими полный цикл эволюционного процесса с использованием кодов Прюфера в качестве представления решений. Применение кодов Прюфера обеспечило корректность всех генерируемых решений, исключив необходимость проверок на ацикличность деревьев. Двухуровневая функция приспособленности с системой штрафов обеспечивает эволюцию от невалидных решений к оптимальным. Алгоритм поддерживает гибкую настройку параметров, включая типы селекции, скрещивания и мутации, механизм элитизма и адаптивный критерий сходимости.

Слой работы с данными построен на основе pydantic-схем, обеспечивающих валидацию и целостность данных. Классы Graph, PruferCode, GraphGenerator и FileIO предоставляют полный функционал для работы с графами, их генерации и загрузки. Класс GraphManager объединяет функциональность в единый интерфейс.

Графический интерфейс, разработанный с использованием tkinter, matplotlib и networkx, предоставляет интуитивный способ взаимодействия с системой. Интерфейс разделён на специализированные компоненты: ParameterFrame для настройки параметров, ControlButtons для управления эволюцией, StatisticsFrame для отображения метрик и графика сходимости. Система поддерживает три способа создания графов: загрузку из файла, генерацию случайных графов и ручной ввод рёбер. Класс GraphDrawer обеспечивает визуализацию с выделением рёбер МОД и отображением любой особи из популяции. Интерфейс предоставляет пошаговое выполнение, автоматическое проигрывание и быстрый переход к результату.

Разработанная система успешно прошла тестирование на графах различной сложности. Модульная архитектура и использование современных практик разработки обеспечивают поддерживаемость и расширяемость системы. Полученные результаты подтверждают эффективность применения генетических алгоритмов для решения задачи поиска МОД. Система может использоваться в образовательных целях для изучения эволюционных алгоритмов и в качестве основы для решения практических задач оптимизации сетевых структур.

Исходный код проекта: <https://github.com/goormany/GeneticAlgorithms>

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 28.04.2021).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.
3. Вирсански Э. В52 Генетические алгоритмы на Python / пер. с англ. А. А. Слинкина. – М.:
4. ДМК Пресс, 2020. – 286 с.: ил. Панченко, Т. В. Генетические алгоритмы [Текст] : учебно-методическое пособие / под ред. Ю. Ю. Тарасевича. — Астрахань : Издательский дом «Астраханский университет», 2007. — 87 [3] с.
5. Tkinter Documentation. Python Software Foundation. – Режим доступа: <https://docs.python.org/3/library/tkinter.html>
6. Pydantic Documentation. – Режим доступа: <https://docs.pydantic.dev/>
7. NetworkX Documentation. – Режим доступа: <https://networkx.org/documentation/stable/>